

LABORATORIO DI SISTEMI OPERATIVI

System Calls for I/O
Standard I/O Library

fabio.rossi@unipg.it

075 5855005

Accesso ai file (ed altro...)

- Per l'accesso ai file vi sono due "possibilità":
 - System Calls (*open(2)*, *write(2)*, etc.)
 - Standard I/O Library (*fopen(3)*, *fwrite(3)*, etc.)

System Calls per i file

Nome	Cosa fa
open	apre un file in lettura e/o scrittura o crea un nuovo file
close	chiude un file precedentemente aperto
read	legge da un file
write	scrive su un file
lseek	sposta il puntatore di lettura/scrittura ad una determinata posizione nel file
unlink	rimuove una entry di un file nel filesystem
stat (fstat, lstat)	fornisce informazioni sugli attributi di un file
dup, dup2, dup3	duplicano un file descriptor

System Calls per le directory

Nome	Cosa fa
mkdir	crea una directory
rmdir	cancella una directory (se vuota)
chdir	cambia la directory corrente

Open ([man 2 open](#))

```
#include <fcntl.h>
int open(const char *pathname,
        int flags,
        mode_t mode)
```

- apre il file specificato da *pathname*
- *flags* **deve** includere uno tra *O_RDONLY*, *O_WRONLY* o *O_RDWR*
- *mode* specifica i permessi da applicare al file in caso di creazione (in realtà i permessi effettivi sono stabiliti in base anche ad [UMASK](#)). La creazione può essere richiesta specificando *O_CREAT* tra i *flags*, altrimenti *mode* viene ignorato e può essere omesso

Altri *flags* (classicamente separati da |):

- *O_CREAT*: se il file non esiste lo crea
- *O_APPEND*: il file è aperto in accodo (per la scrittura)
- *O_TRUNC*: se il file esiste il contenuto viene azzerato

Open

- valori di ritorno:
 - un intero positivo che è il **file descriptor*** da utilizzare per le successive operazioni (*close*, *write*, *read*, etc.). Oppure:
 - -1 in caso di errore (viene settata [errno](#))
- di default il puntatore (file offset) nel file è posizionato all'inizio (a meno che, per esempio, non si specifichi `O_APPEND...`)
- I file aperti dovrebbero essere chiusi prima di terminare il programma con:
`#include <unistd.h>`
`int close(int fd)`
dove *fd* è il file descriptor tornato da *open()*. Ritorna 0 o -1 in caso di errore (setta *errno*)
- l'apertura di un file crea una nuova "entry" nella tabella di sistema dei file aperti (vedi anche slide successiva...): "A call to **`open()`** creates a new **open file description**, an entry in the system-wide table of open files. The open file description records the file offset and the file status flags. A file descriptor is a **reference** to an open file description" (da man 2 open)

Open file descriptions (*man 2 open:NOTES*)

- The term open file description is the one used by *POSIX* to refer to the entries in the system-wide table of open files. In other contexts, this object is variously also called an "*open file object*", a "*file handle*", an "*open file table entry*", or—in kernel-developer parlance—a *struct file*.
- When a file descriptor is duplicated (using *dup(2)* or similar), the duplicate refers to the same open file description as the original file descriptor, and the two file descriptors consequently share the *file offset* and *file status flags*. Such sharing can also occur between processes: a *child* process created via *fork(2)* inherits duplicates of its parent's file descriptors, and those duplicates refer to the same open file descriptions.
- Each *open()* of a file creates a new open file description; thus, there may be multiple open file descriptions corresponding to a file *inode*.
- On *Linux*, one can use the *kcmp(2) KCMP_FILE* operation to test whether two file descriptors (in the same process or in two different processes) refer to the same open file description.

Write, read

```
#include <unistd.h>
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

Scrive i primi *count* byte contenuti in *buf* nel file aperto identificato da *fd* (*file descriptor* ritornato da *open(2)*).

Ritorna il numero di byte scritti (che può anche essere inferiore a *count* se, per esempio, il device di scrittura si è riempito) o -1 in caso di errore (*errno* settata)

```
ssize_t read(int fd, void *buf, size_t count);
```

Legge *count* byte dal file identificato da *fd* (*file descriptor* ritornato da *open(2)*) e li memorizza in *buf*.

Ritorna il numero di byte letti (che può anche essere inferiore a *count* se c'erano a disposizione nel device di lettura meno byte) o -1 in caso di errore (*errno* settata). Il valore di ritorno 0 significa *EOF* (*end of file*).

N.B.: Le operazioni di lettura/scrittura sono eseguite a partire dalla posizione corrente (*puntatore*, *cursore* o *file offset*) nel file

lseek

```
#include <unistd.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

riposiziona il puntatore nel file. A seconda dei seguenti valori di *whence* il puntatore è posizionato:

- *SEEK_SET* : a *offset* byte dall'inizio
- *SEEK_CUR*: a *offset* byte dalla posizione corrente
- *SEEK_END*: a *offset* byte dopo la fine del file

Ritorna la nuova posizione (distanza in byte) da inizio file o -1 in caso di errore (setta *errno*)

Esempio

```
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>
#include <stdio.h>
#include <errno.h>
#define ERRORE(s, b) { sprintf(b, "%s, errno = %d", s, errno); perror(b);
exit(EXIT_FAILURE); }

int main()
{
    int fd, iBytesWritten;
    char sBuffer[] = "Scrivi questo!", sError[128], sFileName[] = "myfile";
    /* notate con quali permessi viene creato il file... */
    fd = open(sFileName, O_CREAT|O_TRUNC|O_RDWR, S_IRUSR|S_IWUSR|S_IRGRP|S_IWGRP);
    if(fd > 0) {
        if((iBytesWritten = write(fd, sBuffer, sizeof(sBuffer))) != sizeof(sBuffer))
            ERRORE("Write error", sError);

        lseek(fd, 0, SEEK_SET); /* Il puntatore torna a inizio file */
        if(read(fd, sBuffer, iBytesWritten) != -1)
            printf("Ho (ri)letto: %s\n", sBuffer);
        else
            ERRORE("Read error", sError);
        close(fd);
    }
    else {
        ERRORE("Open error", sError);
    }
    return EXIT_SUCCESS;
}
```

dup, dup2

```
#include <unistd.h>
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

- *dup()* torna una copia di *oldfd* usando il primo file descriptor libero (minore)
- *dup2()* torna una copia di *oldfd* ma usa *newfd* come file descriptor copia.
N.B.: Se c'era già un file aperto con descrittore pari a newfd viene chiuso
- dopo la duplicazione i due file descriptor possono essere usati indifferentemente, **ma si riferiscono alla stessa *entry* nella tabella di sistema che elenca i file aperti** e quindi, per esempio, condividono la posizione corrente nel file (se viene modificata tramite uno dei due la modifica vale anche per l'altro)
- Ritornano il nuovo file descriptor copia; o -1 in caso di errore (*errno* settata)

Esempio

```
fd = open("myfile",  
          O_WRONLY | O_CREAT | O_TRUNC,  
          S_IWUSR | S_IRUSR);  
  
if (dup2(fd, STDOUT_FILENO) != STDOUT_FILENO)  
    perror("dup2"), exit(1);  
  
printf("Ciao!\n");  
  
close(fd);
```

Altre risorse

- [Qui](#) trovate interessanti informazioni sulle Syscalls per i file (*Copyright ©: University of Illinois CS 241 Staff*)
- da [Linux Assembly](#)

Standard I/O Library

- la *standard I/O library* è un insieme di funzioni (**man 3 stdio**) per accesso ai file, agli *streams* standard, etc. che astraggono le operazioni di input-output sotto forma di *streams*
- le operazioni sono **bufferizzate in spazio utente** e ciò riduce il numero di chiamate alle sottostanti *system calls* ed i relativi cambi di contesto ottimizzando le prestazioni
- forniscono un'interfaccia verso il S.O. di più alto livello rispetto alle *System Calls*

Standard I/O library

System Call (no bufferizzazione)	StdIO Library (bufferizzazione)
open	fopen
close	fclose
read/write	getchar/putchar, getc/putc, fgetc/fputc, fread/fwrite, gets/puts, fgets/fputs, scanf/printf, fscanf/fprintf
lseek	fseek

SysCalls vs Standard I/O Library

- **open(),read(),write(),close(), ...**
sono ***System Calls***
- non bufferizzate in spazio utente
- richiedono cambio di contesto user-kernel

- **fopen(), fread(), fwrite(), fclose(), ...**
standard I/O Library del C
- bufferizzano nello spazio utente
- tipicamente più efficienti e veloci